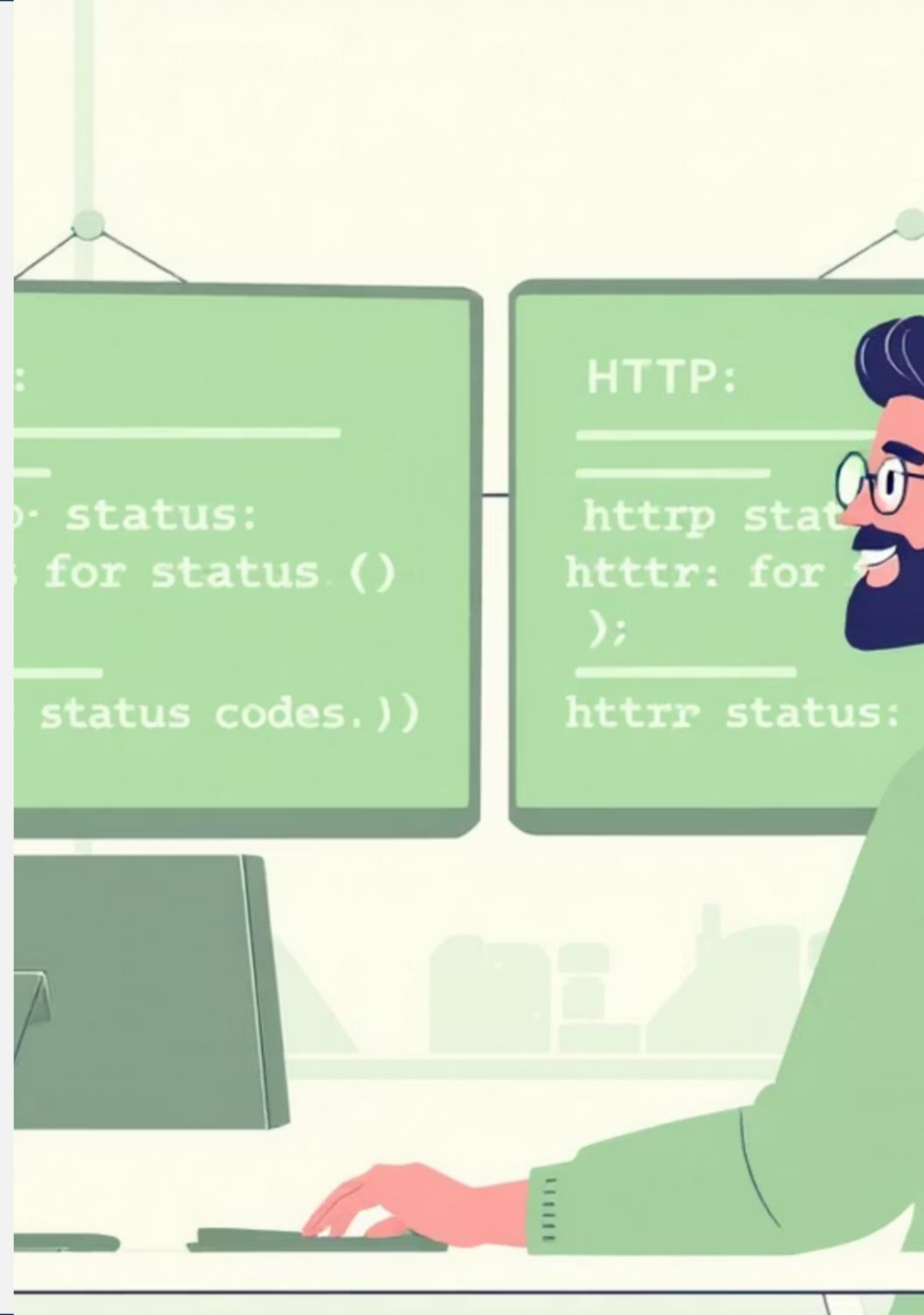


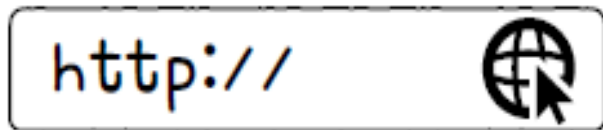
HTTP Message

- HTTP의 특성과 메시지 구조
- 메서드와 상태 코드



HTTP의 중요한 네 가지 특성

- 1) 요청과 응답을 기반으로 동작
- 2) 미디어 독립적
- 3) 상태를 유지하지 않음
- 4) 지속 연결을 지원



HTTP의 네 가지 특성을
기반으로 HTTP를 학습해
봅시다!



HTTP: 미디어 독립적 프로토콜

- HTTP를 정의한 공식 문서(RFC 9110)
 - 자원 - HTTP가 요청하는 대상
 - HTTP는 자원의 특성을 제한하지 않으며, 단지 자원과 상호 작용하는 데 사용할 수 있는 인터페이스를 정의
 - 대부분의 자원은 URI로 식별

RFC 9110

The target of an HTTP request is called a “resource”. HTTP does not limit the nature of a resource; it merely defines an interface that might be used to interact with resources. Most resources are identified by a Uniform Resource Identifier (URI).

MIME 타입 분류



Application

application/pdf, application/zip



Audio

audio/mpeg, audio/wav



Image

image/jpeg, image/png



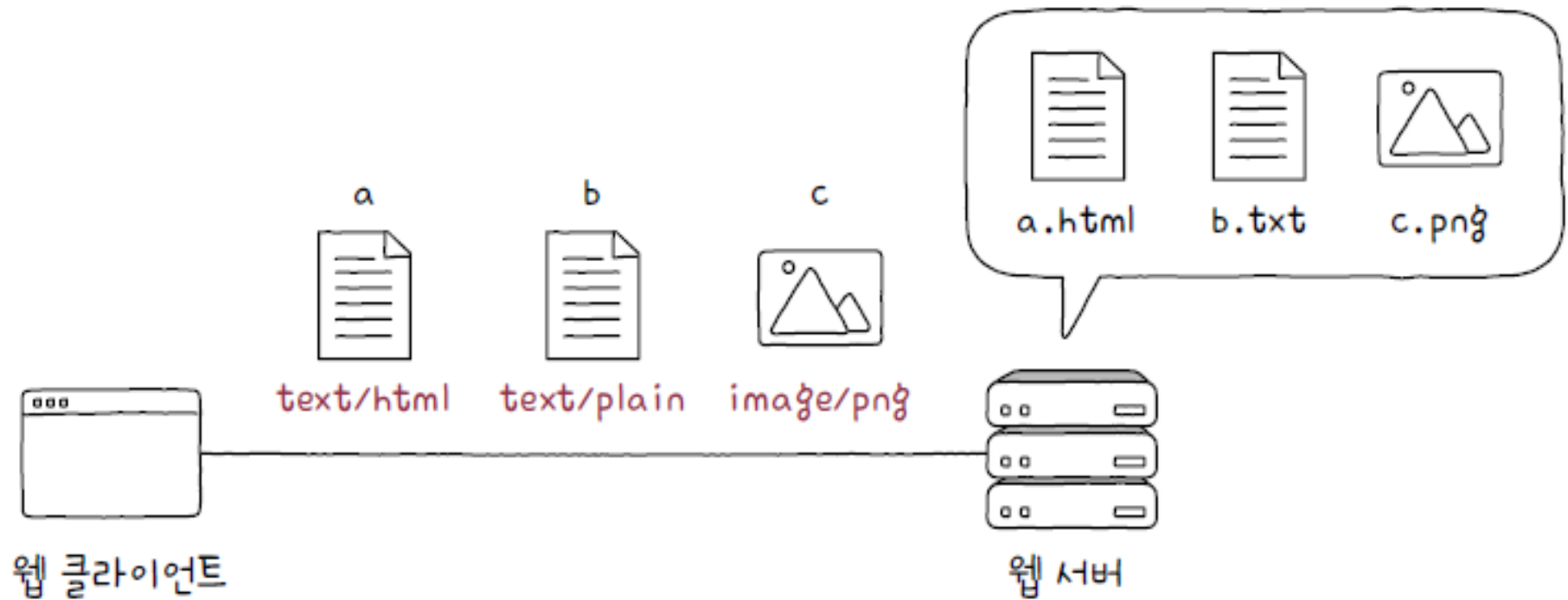
Video

video/mp4, video/webm

미디어 타입(Media Type)

- HTTP에서 메시지로 주고받는 자원의 종류

- MIME 타입(Multipurpose Internet Mail Extensions Type)이라고도 함
- HTTP는 주고받을 미디어 타입에 특별히 제한을 두지 않고 독립적으로 동작이 가능한 미디어 독립적인 프로토콜



MIME 타입 구조

기본 구조

type/subtype

- type: 일반 카테고리 (video, text 등)
- subtype: 정확한 데이터 종류

매개변수 포함

type/subtype;parameter=value

예: text/plain;charset=UTF-8

MIME

미디어 타입의 구성과 종류

- 슬래시를 기준으로 하는 ‘타입/서브타입(type/subtype)’ 형식으로 구성
 - 타입(type) – 데이터의 유형
 - 서브타입(subtype) – 주어진 타입에 대한 세부 유형
- 미디어 타입의 종류는 매우 다양하며, 필요에 따라 새로운 미디어 타입을 등록할 수도 있음
 - 따라서 모든 미디어 타입을 암기할 필요는 없고, 필요할 때마다 찾아보는 것이 일반적
- 미디어 타입에는 추가적인 설명을 위해 선택적으로 매개변수가 포함매개변수
 - ‘타입/서브타입;매개변수=값’의 형식으로 표현

type/subtype

type/subtype;parameter=value

대표적인 미디어 타입과 서브타입(1)

타입	타입 설명	서브타입	서브타입 설명
text	일반 텍스트 형식의 데이터	text/plain	평문 텍스트 문서
		text/html	HTML 문서
		text/css	CSS 문서
		text/javascript	자바스크립트 문서
image	이미지 형식의 데이터	image/png	PNG 이미지
		image/jpeg	JPEG 이미지
		image/webp	WebP 이미지
		image/gif	GIF 이미지
video	비디오 형식의 데이터	video/mp4	MP4 비디오
		video/ogg	OGG 비디오
		video/webm	WebM 비디오

대표적인 미디어 타입과 서브타입(2)

타입	타입 설명	서브타입	서브타입 설명
audio	오디오 형식의 데이터	audio/midi	MIDI 오디오
		audio/wav	WAV 오디오
application	바이너리 형식의 데이터	application/octet-stream	알 수 없는 바이너리 데이터를 포함한 일반적인 바이너리 데이터
		application/pdf	PDF 문서 형식 데이터
		application/xml	XML 형식 데이터
		application/json	JSON 형식 데이터
		application/x-www-form-urlencoded	HTML 입력 폼 데이터(키-값 형태의 입력값을 URL 인코딩한 데이터)
multipart	각기 다른 미디어 타입을 가질 수 있는 여러 요소로 구성된 데이터	multipart/form-data	HTML 입력 폼 데이터
		multipart/encrypted	암호화된 데이터

Multipart 타입

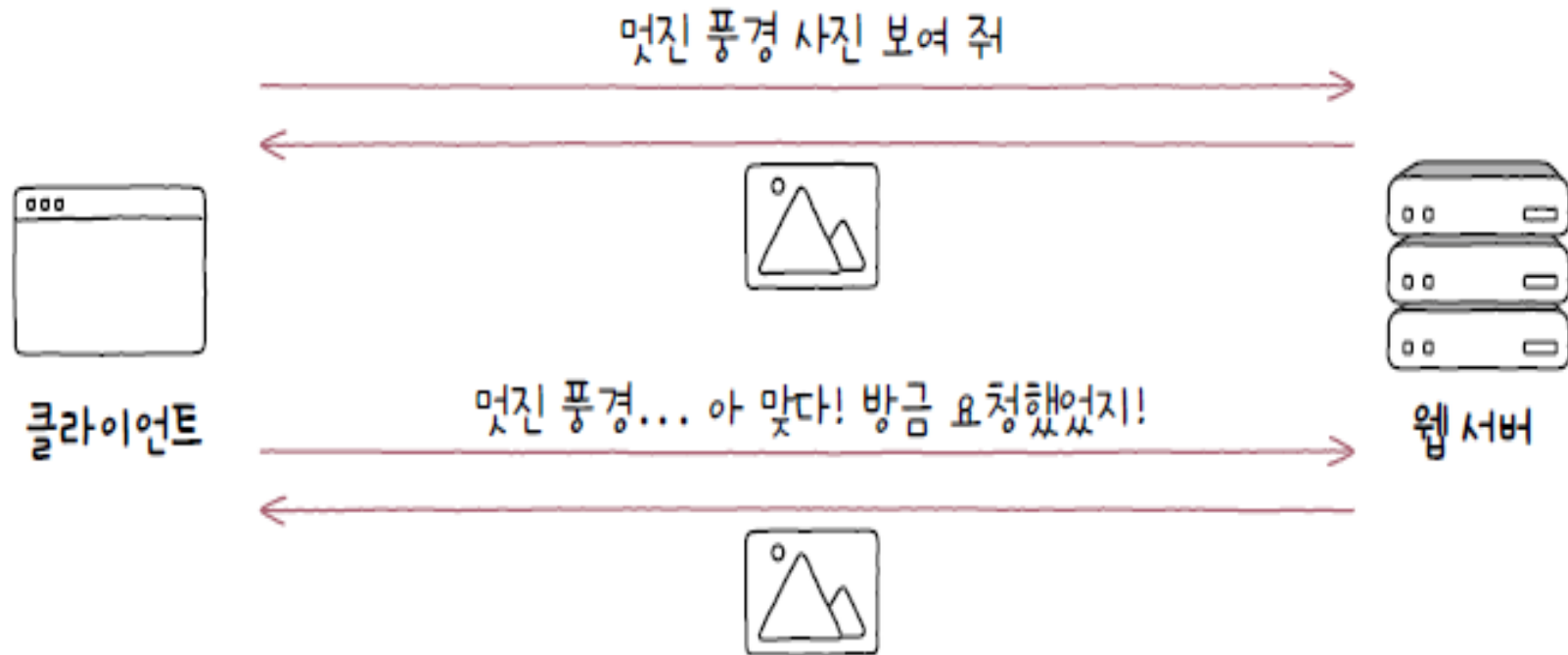
여러 타입의 문서를 포함할 수 있으며, 경계는 --로 시작하는 문자열로 구분됩니다.

```
Content-Type: multipart/form-data;  
boundary=aBoundaryString
```

```
--aBoundaryString  
Content-Disposition: form-data;  
name="myFile";  
filename="img.jpg"  
Content-Type: image/jpeg  
(data)  
--aBoundaryString--  
(more subparts)  
--aBoundaryString--
```

HTTP: 스테이트리스 프로토콜

- HTTP는 상태를 유지하지 않는 스테이트리스(stateless) 프로토콜
 - 서버가 HTTP 요청을 보낸 클라이언트와 관련된 상태를 기억하지 않는다는 의미
 - 클라이언트의 모든 HTTP 요청은 기본적으로 독립적인 요청으로 간주



HTTP: 스테이트리스 프로토콜

- 상태를 유지하지 않는 특성의 장점

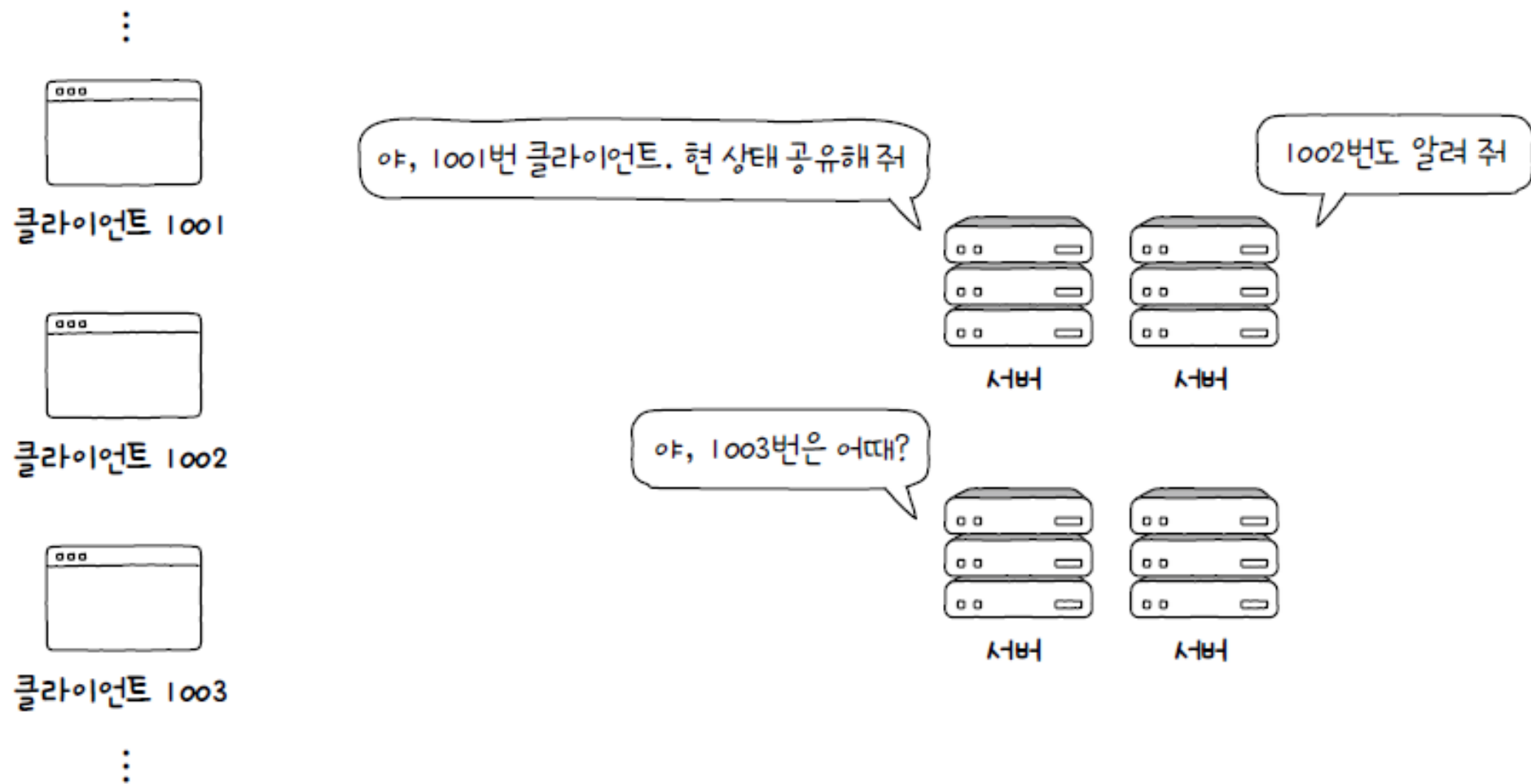
- HTTP 서버는 일반적으로 많은 클라이언트와 동시에 상호 작용

- 동시에 처리해야 할 요청 메시지의 수는 수천 개가 될 수도 있고, 많게는 수백만 개
- 모든 클라이언트의 상태 정보를 유지하는 것은 서버에 큰 부담

- 서버는 하나가 아니라 여러 대로 구성될 경우

- 모든 서버가 모든 클라이언트의 상태를 유지할 경우 클라이언트는 여러 서버를 동시에 이용하기가 어려워짐
- 서버가 모든 클라이언트의 상태 정보를 공유하는 작업은 매우 번거롭고 복잡

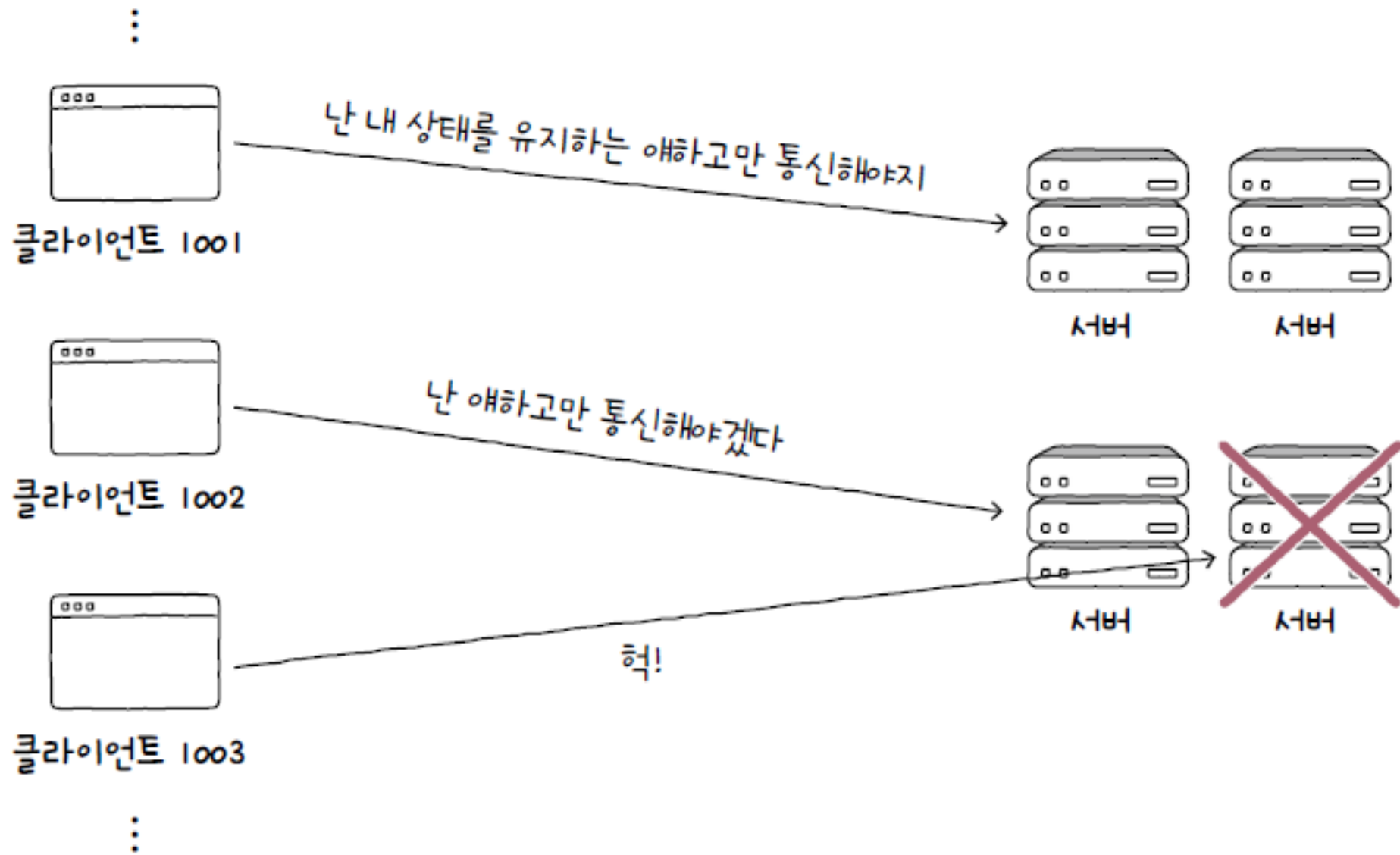
HTTP: 스테이트리스 프로토콜



HTTP: 스테이트리스 프로토콜

- 특정 클라이언트가 특정 서버에 종속되는 상황 방지
 - HTTP가 상태를 유지하는 프로토콜이었다면 클라이언트는 자신의 상태를 기억하는 특정 서버하고만 상호 작용할 수 있게 되어,
특정 클라이언트가 특정 서버에 종속될 수 있음
 - 이러한 상황에서 어느 한 서버에 문제가 발생하면 해당 서버에 종속된 클라이언트는 직전까지의 HTTP 통신 내역을 잃어버리는 상황이 발생
- 확장성(scalability)과 견고성(robustness)
 - HTTP가 처음 만들어졌을 때부터 오늘날까지 이어지는 중요한 설계 목표

HTTP: 스테이트리스 프로토콜



HTTP : 지속 연결 프로토콜

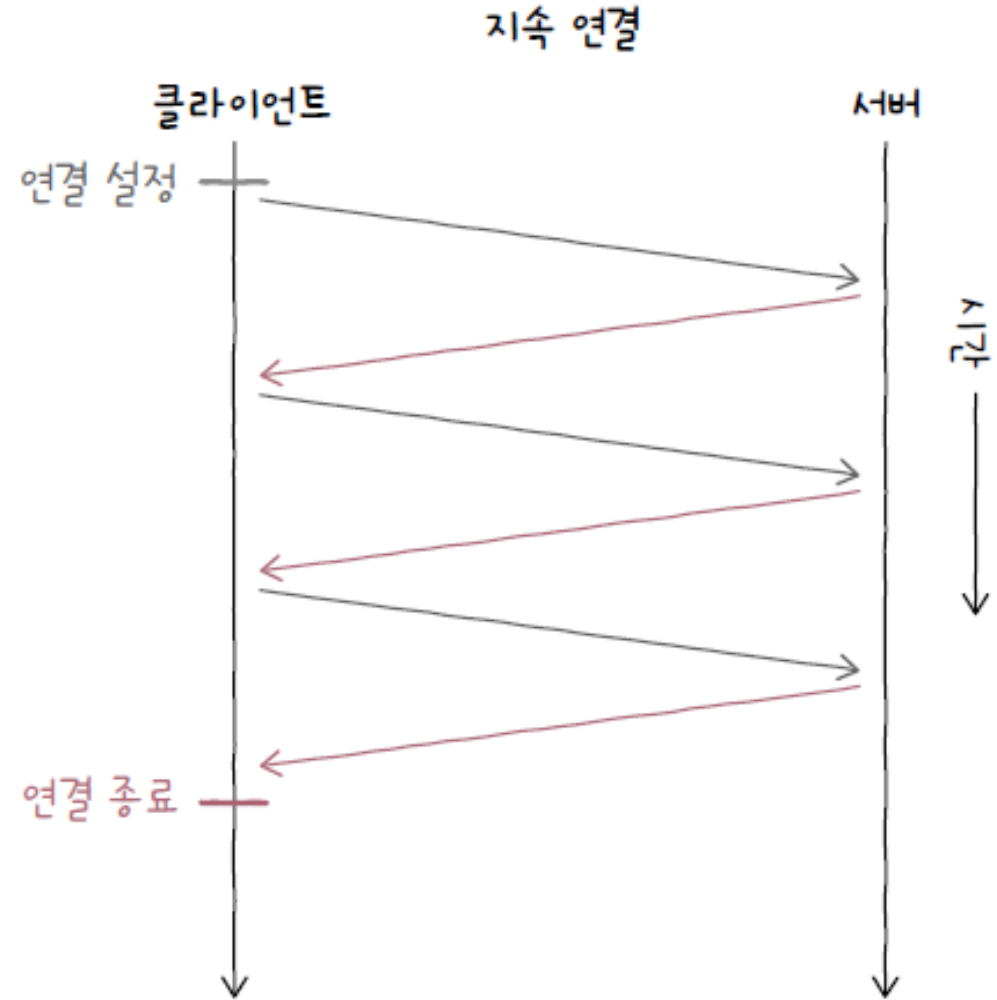
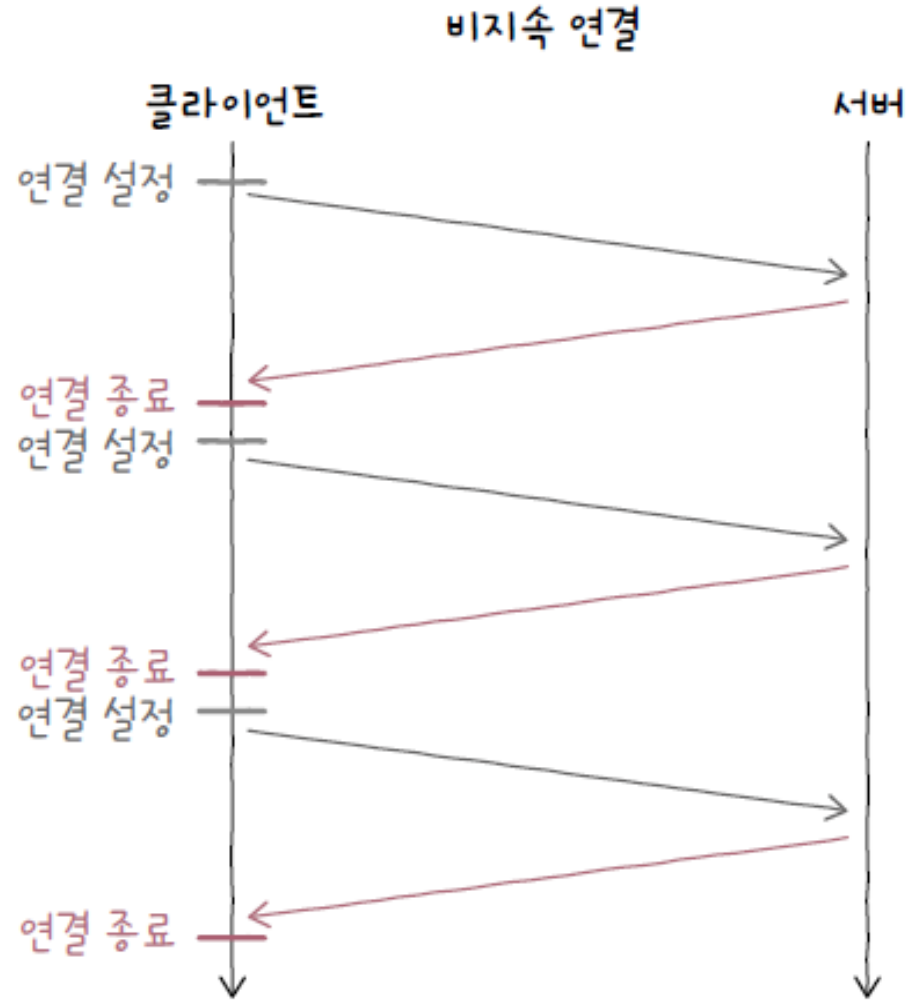
- 비지속 연결

- 초기의 HTTP 버전(HTTP 1.0 이하)은 3-way 핸드셰이크를 통해 TCP 연결을 수립한 후, 요청에 대한 응답을 받으면 연결을 종료하는 방식으로 동작
- 추가적인 요청-응답을 하기 위해서는 다시 TCP 연결을 수립이라고 합니다

- 지속 연결(persistent connection) 또는 킵 얼라이브(keep-alive)

- 최근 대중적으로 사용되는 HTTP 버전(HTTP 1.1 이상)
- 하나의 TCP 연결상에서 여러 개의 요청-응답을 주고받을 수 있는 기술

Chapter 05-2 HTTP(15)



HTTP 통신 구조 이해

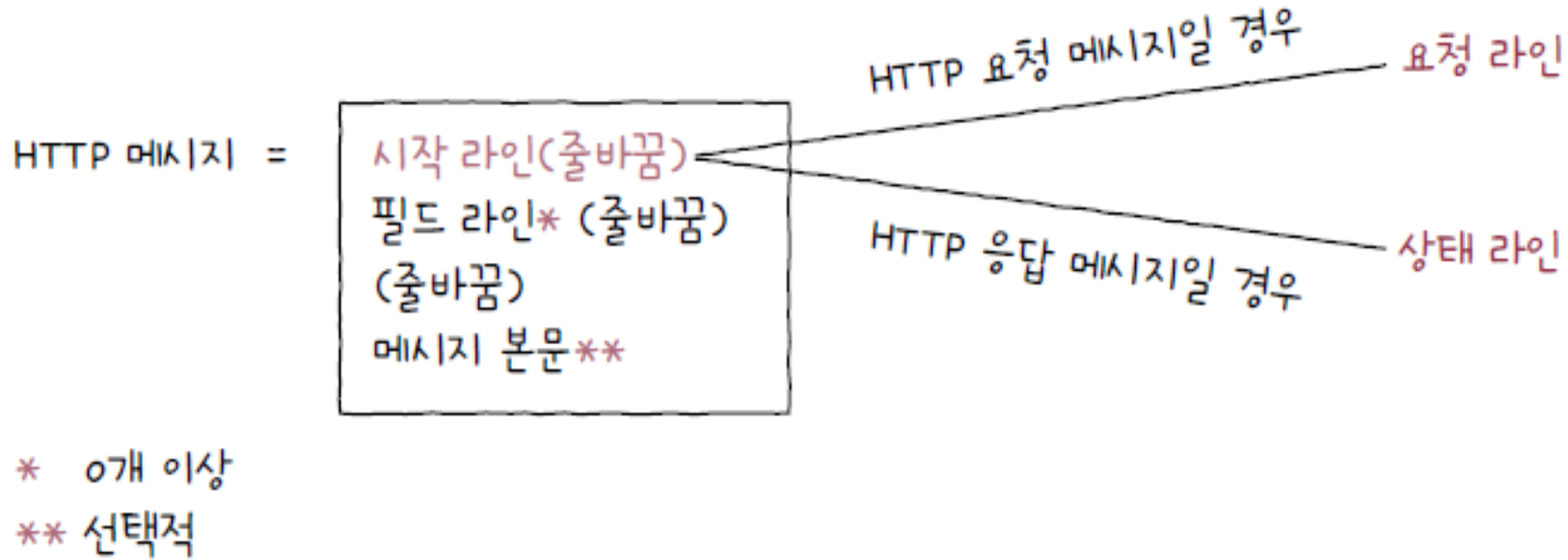
HTTP 통신은 클라이언트와 서버 간의 요청-응답 모델로 동작합니다. 클라이언트가 요청을 보내면 서버가 상태 코드와 함께 응답을 반환합니다.

- 요청 메시지 구조
- 응답 메시지 구조
- 상태 코드의 역할

HTTP 메시지 구조

◦ 시작 라인(start-line)

- HTTP 메시지는 HTTP 요청 메시지일 수도 있고, HTTP 응답메시지일 수도 있음
 - HTTP 메시지가 HTTP 요청 메시지일 경우 시작 라인은 '요청 라인'이 되고,
 - HTTP 메시지가 HTTP 응답 메시지일 경우 시작 라인은 '상태 라인'이 됨



HTTP 요청 메시지 구조

Method Path Protocol version

HTTP 메시지 =

시작 라인(줄바꿈)
필드 라인* (줄바꿈)
(줄바꿈)
메시지 본문**

* 0개 이상

** 선택적

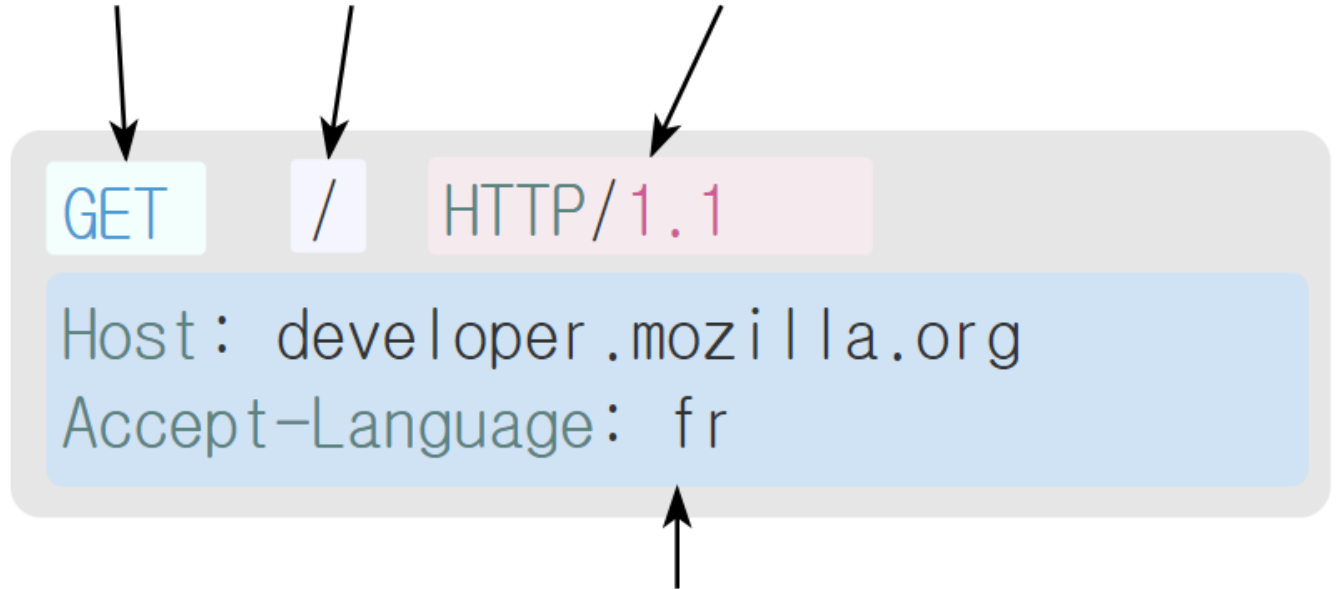
HTTP 요청 메시지는

시작줄(Request Line),

헤더(필드라인, 0개~여러 개)

빈 줄바꿈

본문(Body, 생략가능)으로 구성됩니다.



Headers

요청 라인 = 메서드 (공백) 요청 대상 (공백) HTTP 버전 (줄바꿈)

주요 HTTP 메서드 (1/2)

메서드(method)

- 클라이언트가 서버의 자원(요청 대상)에 대해 수행할 작업의 종류
- 대표적으로 GET, POST, PUT, DELETE 등

GET

URL에 해당하는 리소스를 가져오는 요청입니다.

POST

HTML 폼 데이터를 서버로 전송하며, 멍등성을 보장하지 않습니다.

HEAD

GET 요청의 헤더 부분만 요청합니다.

PUT

지정한 URL에 데이터를 저장하는 요청입니다.

멍등성이란? <https://developer.mozilla.org/ko/docs/Glossary/Idempotent>

주요 HTTP 메서드 (2/2)

PATCH

URL의 데이터를 부분적으로 수정하는 요청입니다.

DELETE

지정한 URL의 정보를 제거하는 요청입니다.

TRACE

송신한 요청의 내용을 반환해주는 요청입니다.

OPTIONS

해당 URL에서 지원하는 메서드 목록을 요청합니다.

PATH: 요청 대상(request-target)

- HTTP 요청을 보낼 서버의 자원
- 보통 (쿼리가 포함된) URI의 경로가 명시
 - 예) 클라이언트가 “http://www.example.com/hello?q=world”로 요청을 보낼 경우, 요청 대상은 “/hello?q=world”
 - 만약 하위 경로가 없더라도 요청 대상은 슬래시(/)로 표기
 - 예) 클라이언트가 “http://www.example.com”으로 요청할 경우 요청 대상은 “/”

HTTP 버전(HTTP-version)

- 사용된 HTTP 버전
- ‘HTTP/〈버전〉’이라는 표기 방식을 따르며, HTTP 버전 1.1은 HTTP/1.1로 표기

◦ 필드 라인 또는 헤더 라인(header-line)

- 0개 이상의 HTTP 헤더 HTTP header가 명시
- HTTP 헤더 - HTTP 통신에 필요한 부가 정보를 의미
- 콜론(:)을 기준으로 헤더 이름(header-name)과 하나 이상의 헤더값(header-value)으로 구성

```
GET /example-page HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101
```

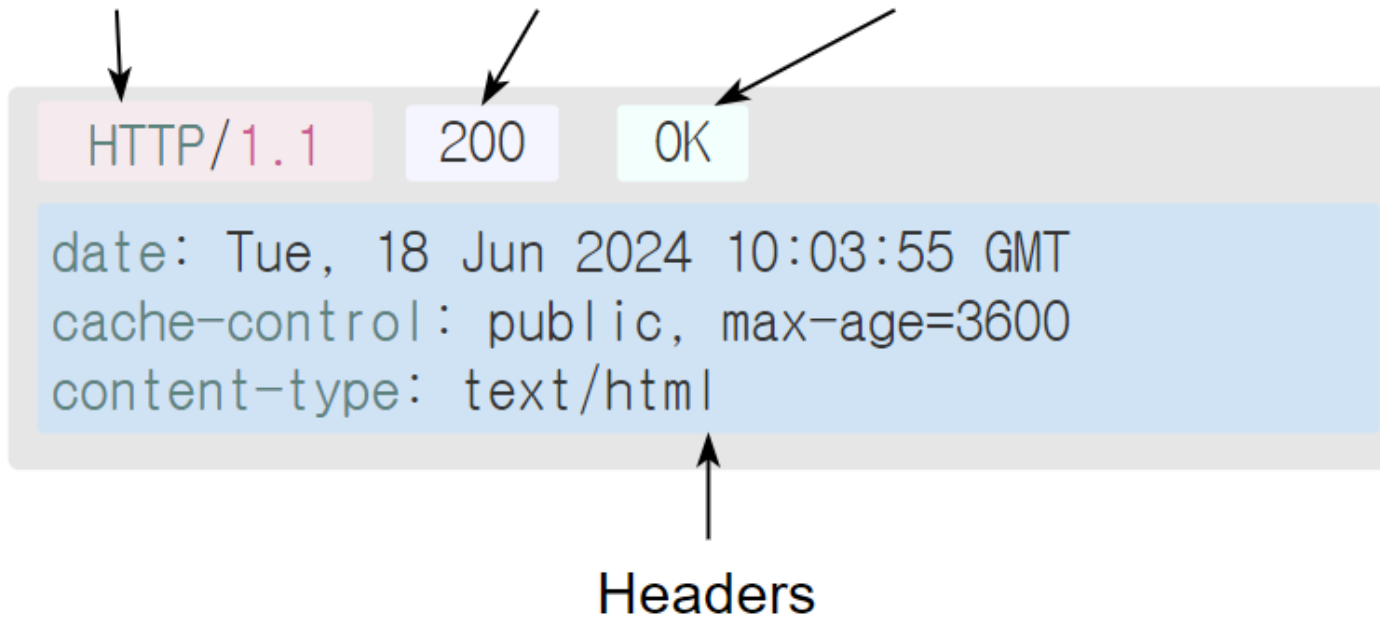
```
Firefox/118.0
```

```
Accept: text/html
```


HTTP 응답 메시지 구조

HTTP 응답 메시지는 상태 코드가 포함된 상태 라인으로 시작됩니다.

Protocol version Status code Status message



HTTP/1.1 200 OK

HTTP/1.1 404 Not Found

- 상태 코드 - 요청에 대한 결과를 나타내는 세 자리 정수
- 이유 구문 - 상태 코드에 대한 문자열 형태의 설명

HTTP 상태 코드 5가지 주요 분류

1xx 정보 전송

임시 응답으로 클라이언트에게 계속 진행하라는 의미.
HTTP 1.1부터 추가됨.

2xx 성공

요청이 성공적으로 수신되고 처리되었음을 의미.

3xx 리다이렉션

추가 동작이 필요한 경우. 주로 주소 이동을 알림.

4xx 클라이언트 에러

클라이언트 요청에 문제가 있거나 금지된 접근.

5xx 서버 에러

서버 문제로 인한 처리 실패. 나중에 재시도 시 해결 가능.

핵심 HTTP 상태 코드 상세 분석

1

200 OK

클라이언트의 HTTP 요청이 서버에서 완전히 받아들여져 처리에 성공했음을 나타냅니다.

2

301 Moved Permanently

요청한 URI가 완전히 다른 주소로 이동. Location 헤더에 새 주소 포함.

3

304 Not Modified

지정된 시각 이후 변화가 없어 로컬 캐시 사용. If-Modified-Since 헤더 참조.

◦ 메시지 본문(message-body)

- HTTP 요청 혹은 응답 메시지에서 본문이 필요할 경우 이는 메시지 본문에 명시
- 메시지 본문은 존재하지 않을 수도 있고, 다음과 같이 다양한 콘텐츠 타입이 사용될 수도 있음

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

```
{  
  "id": 12345,  
  "name": "John Doe",  
  "email": "johndoe@example.com"  
}
```

JSON



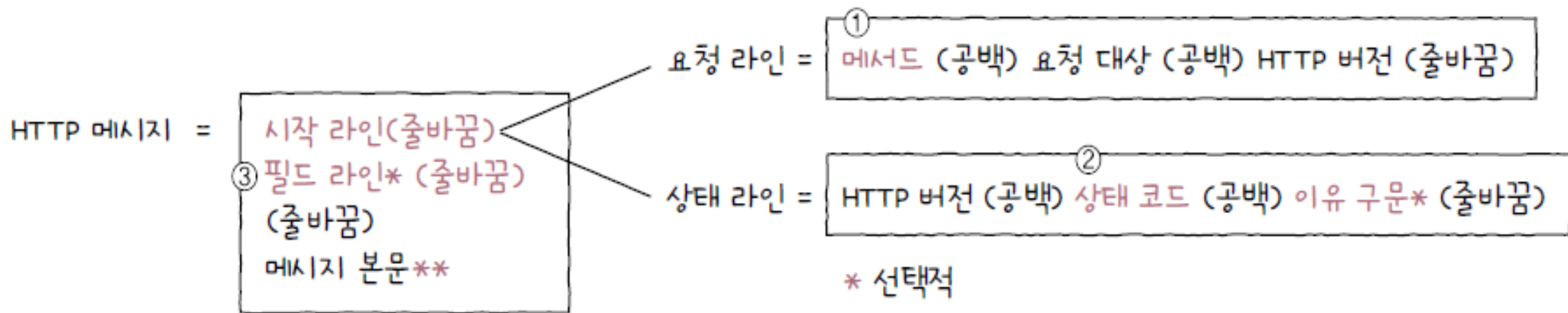
```
HTTP/1.1 200 OK  
Content-Type: text/html
```

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Example HTML Page</title>  
</head>  
<body>  
  <h1>Welcome to our website</h1>  
  <p>This is a sample HTML page.</p>  
</body>  
</html>
```

HTML



HTTP 메시지 정리



* 0개 이상

** 선택적

HTTP 요청 메시지의 요청 라인에는 메서드가 명시되고, HTTP 응답 메시지의 상태 라인에는 상태 코드가 명시됩니다.



HTTP 메시지 예시 GET

GET – 가져다주세요

- 특정 자원을 조회할 때 사용되는 메서드
 - 클라이언트가 서버에게 ‘이것(자원)을 가져다주세요’라고 요청을 보내는 것과 같음
 - 자원은 HTML,, JSON, 이미지 파일이나 일반 텍스트 파일 등

① 요청 메시지

- 예) `http://www.example.com/example-page`에 대한 간략화된 GET 요청 메시지

```
GET /example-page HTTP/1.1
```

```
Host: www.example.com
```

```
Accept: *
```

HTTP 메시지 예시 GET

② 응답 메시지

- GET 요청 메시지가 성공적으로 처리되었다면 이에 대한 응답으로서 요청한 자원을 전달받음

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<!DOCTYPE html>
<html>
<head>
  <title>Example Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

HTTP 메시지 예시 GET

③ 요청 메시지

- GET 요청 메시지에서는 메시지 본문보다 쿼리 문자열이 사용되는 경우가 많음

```
GET /index.html?name1=value1&name2=value2 HTTP/1.1
```

```
Host: www.example.com
```

```
Accept: *
```


HTTP 메시지 예시:HEAD

HEAD – 헤더만 가져다주세요

- HEAD 메서드를 사용하면 서버는 요청에 대한 응답으로 응답 메시지의 헤더만을 반환

① 요청 메시지

- 예) `http://www.example.com/example-page`에 대한 HEAD 요청을 보낸 예시

```
HEAD /example-page HTTP/1.1  
Host: www.example.com  
Accept: *
```

② 응답 메시지

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 1234
```

HTTP 메시지 예시: POST

POST – 처리해 주세요

- 서버로 하여금 특정 작업을 처리하도록 요청하는 메서드
 - 예) `http://example.com/posting`에 접속했을 때의 화면이 다음과 같고, 어떤 클라이언트가 입력 폼에 글을 입력한 뒤, [게시하기] 버튼을 눌렀다고 가정



The illustration shows a web browser window with a title bar. The address bar contains the URL `http://example.com/posting`. The main content area is titled "게시판" (Bulletin Board). It contains two text input fields. The first field has the text "오늘도 즐거운 날입니다" (It's a fun day again today). The second field has the text "재미있는 글 보고 가세요~" (Go see an interesting post~). At the bottom right of the form is a button labeled "게시하기" (Post).

HTTP 메시지 예시: POST

- POST 메서드가 사용
 - 처리할 대상은 흔히 메시지 본문으로 명시



- POST 메서드는 많은 경우 ‘클라이언트가 서버에 새로운 자원을 생성하고자 할 때’ 사용
 - 성공적으로 POST 요청이 처리되어 새로운 자원이 생성되면 서버는 응답 메시지의 Location 헤더를 통해 새로 생성된 자원의 위치를 클라이언트에게 알려 줌
 - 다음 응답 메시지의 붉은색 글자 부분 - 새로 생성된 자원은 /posting/1에서 확인할 수 있다는 의미

응답 메시지

```
HTTP/1.1 201 Created
Content-Type: application/json
Content-Length: 100
Date: Mon, 14 Oct 2024 16:35:00 PST
Location: /posting/1

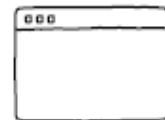
{
  "Id": 1,
  "Title": "오늘도 즐거운 날입니다",
  "Contents": "재미있는 글 보고 가세요~"
}
```

HTTP 메시지 예시: PUT

PUT – 덮어써 주세요

- 요청 자원이 없다면 메시지 본문으로 자원을 새롭게 생성하거나, 이미 자원이 존재한다면 메시지 본문으로 자원을 완전히 대체하는 메서드

예) example.com/posts/1
에 우측 상단과 같은 자원이
있다고 가정 - 이에 대해
좌측 하단과 같이 PUT 요청
메시지(붉은색테두리
박스)를 보낼 경우



클라이언트

```
PUT /posting HTTP/1.1
Host: example.com
...헤더 후략...

{
  "Id": 1,
  "Title": "수정된 제목입니다"
}
```



서버

```
{
  "Id": 1,
  "Title": "오늘도 즐거운 날입니다",
  "Contents": "재미있는 글 보고 가세요~"
}
```

HTTP 메시지 예시: PUT

- PUT 요청은 마치 덮어쓰기와 같으므로 example.com/posts/1의 자원은 다음과 같이 갱신

```
{  
  "Id": 1,  
  "Title": "수정된 제목입니다"  
}
```

HTTP 메시지 예시: PATCH

PATCH - 일부 수정해 주세요

- PUT 메서드가 덮어쓰기, 완전한 대체에 가깝다면 PATCH 메서드는 부분적 수정
 - (아래 화면) 앞 예제에서의 요청 메서드를 PATCH 메서드로 바꿔 보낸 결과
 - PUT 메서드로 요청을 보냈을 경우 메시지 본문으로 덮어써졌지만, PATCH 메서드로 요청을 보낼 경우 메시지 본문에 맞게 자원이 일부 수정

```
{  
  "Id": 1,  
  "Title": "수정된 제목입니다",  
  "Contents": "재미있는 글 보고 가세요~"  
}
```

HTTP 메시지 예시: DELETE

DELETE – 삭제해 주세요

- 특정 자원을 삭제하고 싶을 때 사용하는 메서드
 - 예) example.com/texts/a.txt라는 자원을 삭제하도록 요청하는 메시지

요청 메시지

```
DELETE /texts/a.txt HTTP/1.1
```

```
Host: example.com
```


HTTP 메서드 설계

- 서버 개발자 입장의 메서드 설계

- 어떤 URI(URL)에 어떤 메서드로 요청을 받았을 때 서버가 어떻게 행동해야 하는지 설계하는 것은 오로지 개발자의 몫
 - 어떤 메서드는 구현할 수도 있고, 어떤 메서드는 구현하지 않을 수도 있음
 - 같은 URL에 대한 요청일지라도 사용된 메서드가 다르면 각기 다른 요청으로 간주하기 때문에, 때로는 같은 URL에 대해 메서드별 동작을 여러 개 구현할 수도 있음



◦ API 문서(1)

- 예) 유튜브와 관련된 API

- 어떤 URL에 어떤 메서드를 보낼 수 있는지,
- 어떤 쿼리 문자열(매개변수)이 사용될 수 있는지,
- 올바르게 요청을 보냈을 경우 어떤 응답 메시지를 받을 수 있는 지,
- 그리고 올바르지 않은 요청을 보냈을 경우 어떤 오류 메시지를 받을 수 있는지가 명시

유튜브 API 예시

 YouTube > Data API

요청

HTTP 요청

GET https://www.googleapis.com/youtube/v3/search

매개변수

다음 표에는 이 쿼리가 지원하는 매개변수가 나와 있습니다. 나열된 모든 매개변수는 쿼리 매개변수입니다.

매개변수

필수 매개변수

운영	
list	0개 이상의 리소스 목록을 검색(GET)합니다.
insert	새 리소스를 만듭니다(POST).
update	요청에 포함된 데이터를 반영하도록 기존 리소스를 수정(PUT)합니다.
delete	특정 리소스를 삭제(DELETE)합니다.

사용해 보기

사용해 보기

◦ API 문서(2)

- 예) 네이버의 뉴스 검색 결과를 확인할 수 있는 API
 - 어떤 URL에 어떤 메서드를 보낼 수 있는지,
 - 어떤 쿼리 문자열(매개변수)이 사용될 수 있는지,
 - 어떤 응답 메시지를 받을 수 있는지가 명시

요청 URL	반환 형식
<code>https://openapi.naver.com/v1/search/news.xml</code>	XML
<code>https://openapi.naver.com/v1/search/news.json</code>	JSON

HTTP 상태 코드

- 상태 코드는 요청에 대한 결과를 나타내는 세 자리 정수
 - 상태 코드의 종류는 200, 201, 304, 404, 505 등 다양한데, 백의 자리 수를 기준으로 유형을 구분

상태 코드	설명
100번대(100~199)	정보성 상태 코드
200번대(200~299)	성공 상태 코드
300번대(300~399)	리다이렉션 상태 코드
400번대(400~499)	클라이언트 에러 상태 코드
500번대(500~599)	서버 에러 상태 코드

HTTP 상태 코드

200번대: 성공 상태 코드

- 200번대 상태 코드는 '요청이 성공했음'을 의미
 - 주로 사용되는 상태 코드는 200(OK), 201(Created), 202(Accepted), 204(No Content)

상태 코드	이유 구문	설명
200	OK	요청이 성공했음
201	Created	요청이 성공했으며, 새로운 자원이 생성되었음
202	Accepted	요청을 잘 받았으나, 아직 요청한 작업을 끝내지 않았음
204	No Content	요청이 성공했지만, 메시지 본문으로 표시할 데이터가 없음

HTTP 상태 코드

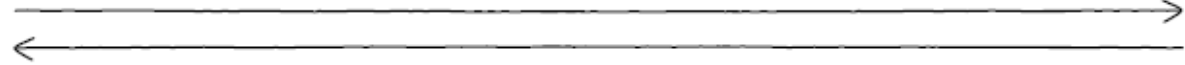
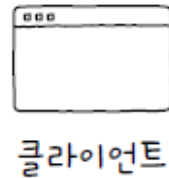
- 예) 클라이언트가 “http://example.com/images/a.png”로 GET 요청을 보냈다고 가정
 - 서버가 이 요청을 성공적으로 받아들이고 처리한 경우, 서버는 요청한 자원과 함께 상태 코드 200(OK)을 포함하여 응답

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<!DOCTYPE html>
<html>
<head>
  <title>Example Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

HTTP 상태 코드

- POST 요청을 통해 서버에 새로운 자원을 생성한 경우, 상태 코드 201(Created)로 요청이 성공했으며 새로운 자원이 만들어졌음을 알림
 - 이 경우 Location 헤더를 통해 생성된 자원의 위치를 명시



```
POST /api/resource HTTP/1.1
Host: example.com
Content-Type: application/Json
Content-Length: 56

{
  "name": "New Resource",
  "data": "Some data for the new Resource"
}
```

```
HTTP/1.1 201 Created
Location: /api/resource/123
Content-Type: application/Json
Content-Length: 32

{
  "Id": 123,
  "name": "New Resource"
}
```

HTTP 상태 코드

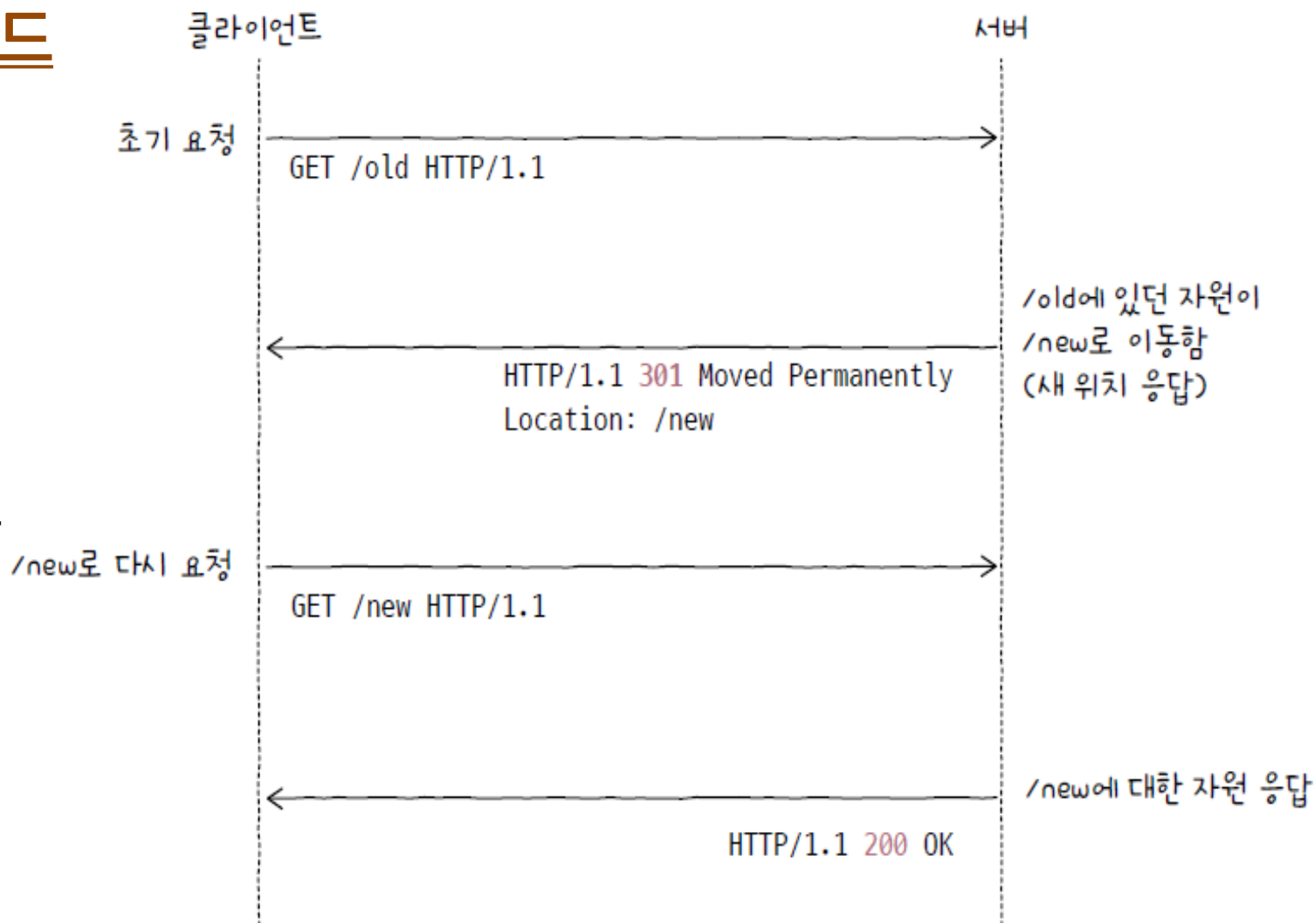
- 202(Accepted)는 요청을 잘 받았으나, 아직 요청한 작업을 끝내지 않았음을 의미
 - 작업 시간이 긴 대용량 파일 업로드 작업이나 배치 작업과 같이 요청 결과를 곧바로 응답하기 어려운 경우
 - 서버는 202(Accepted)로 응답
- 요청 메시지에 대해 성공적으로 작업을 완료했더라도 마땅히 메시지 본문으로 표기할 것이 없을 경우 서버는 상태 코드 204(No Content)로 응답



HTTP 상태 코드

300번대: 리다이렉션 상태 코드

- 리다이렉션(redirection)
 - ‘요청을 완수하기 위해 추가적인 조치가 필요한 상태(인터넷 공식 문서 RFC 9110)’
 - 클라이언트가 요청한 자원이 다른 곳에 있을 때, 클라이언트의 요청을 다른 곳으로 이동시키는 것을 의미



HTTP 상태 코드

- 영구적인 리다이렉션(permanent redirection)(1)
 - 자원이 완전히 새로운 곳으로 이동하여 경로가 영구적으로 재지정
 - 이 경우 기존의 URL에 요청 메시지를 보내면 항상 새로운 URL로 리다이렉트

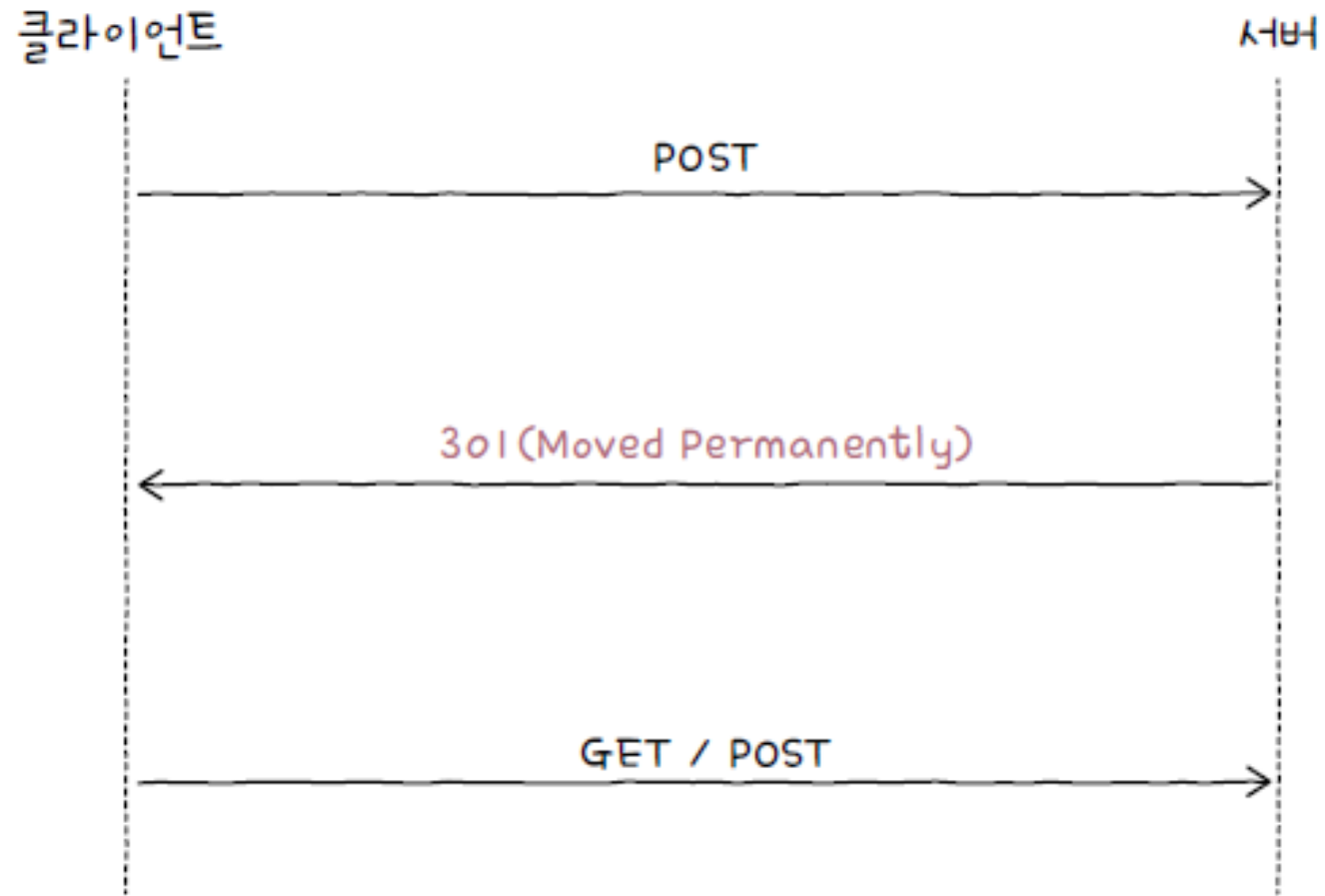
상태 코드	이유 구문	설명
301	Moved Permanently	영구적 리다이렉션; 재요청 메서드 변경될 수 있음
308	Permanent Redirect	영구적 리다이렉션; 재요청 메서드 변경되지 않음

HTTP 상태 코드

◦ 영구적인 리다이렉션(permanent redirection)(2)

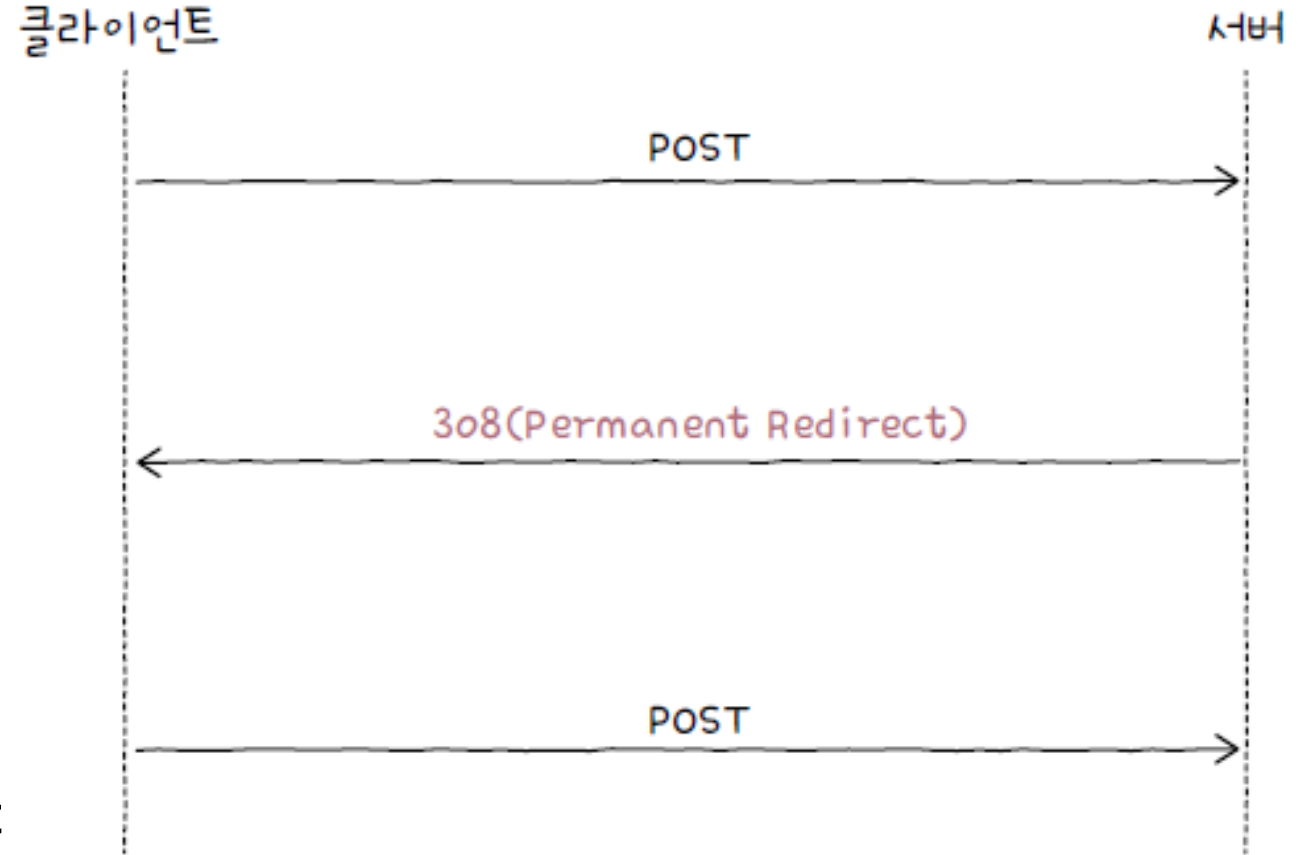
- 1) 클라이언트가 서버에 GET 요청 메시지를 보낸 뒤 301(Moved Permanently) 혹은 308(Permanent Redirect) 응답 메시지를 받았고, 응답 메시지의 Location 헤더에 명시된 경로로 재요청을 보낸다고 가정
 - 클라이언트가 보내는 두 번째 요청 메서드는 첫 번째 요청 메서드와 동일하게 GET
 - 2) 클라이언트가 서버에 POST 메서드와 같이 GET 메서드가 아닌 요청 메시지를 보냈고, 301(Moved Permanently) 응답 메시지를 받았다고 가정
 - 클라이언트가 보내는 두 번째 요청 메서드는 다음 그림처럼 GET 요청으로 바뀔 '수도' 있음
- 실제 공식 문서에서도 이 부분은 'MAY change the request method(요청 메서드가 바뀔 수도 있습니다)'라고 정의되어 있음

HTTP 상태 코드



HTTP 상태 코드

- 영구적인 리다이렉션(permanent redirection)(3)
 - 상태 코드가 308(Permanent Redirect)
 - 클라이언트가 308(Permanent Redirect) 응답 메시지를 받을 경우, 두 번째 요청 메서드는 변하지 않음
 - 첫 번째 요청에서 POST 메서드를 사용했다면, 상태 코드 308(Permanent Redirect)을 받은 뒤 보내는 두 번째 요청에서도 POST 메서드를 유지



HTTP 상태 코드

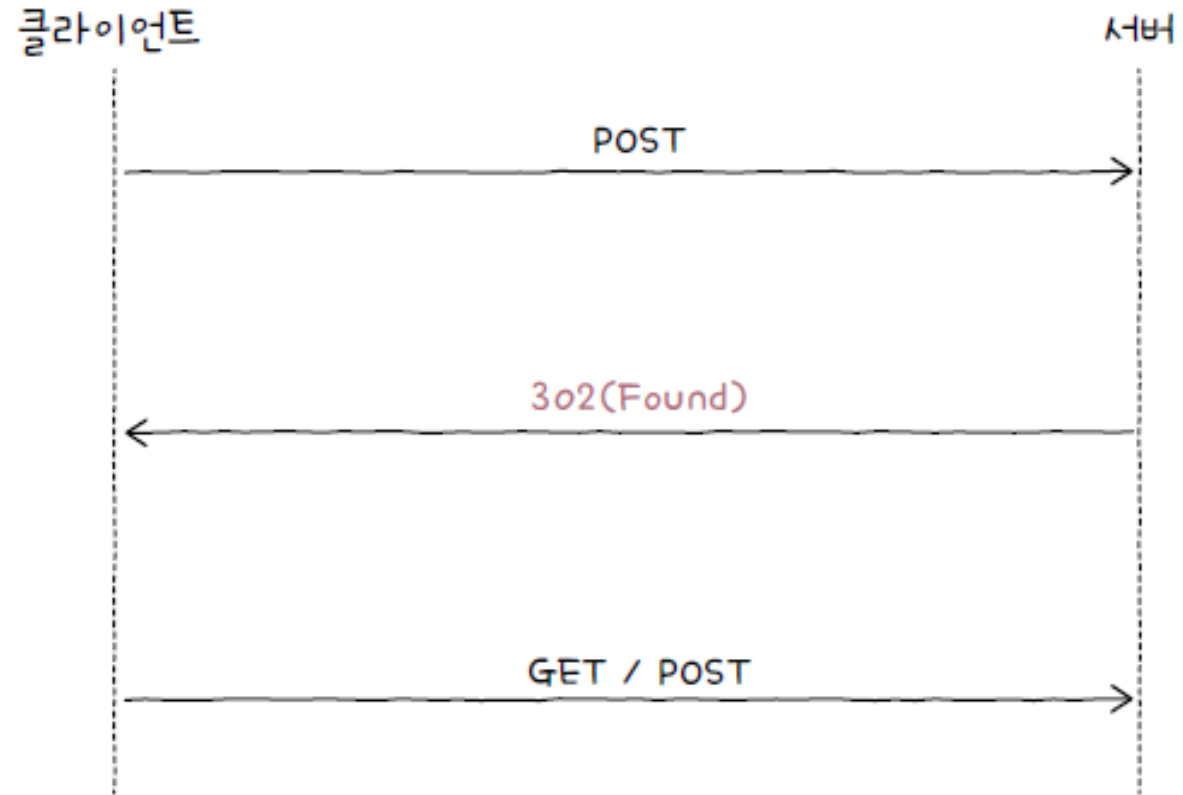
◦ 일시적인 리다이렉션(temporary redirection)(1)

- 자원의 위치가 임시로 변경되었거나 임시로 사용할 URL이 필요한 경우에 주로 사용
 - 어떤 URL에 대해 일시적인 리다이렉션 관련 상태 코드를 응답받았다면 여전히 요청을 보낸 URL은 기억

상태 코드	이유 구문	설명
302	Found	일시적 리다이렉션; 재요청 메서드 변경될 수 있음
303	See Other	일시적 리다이렉션; 재요청 메서드 GET으로 변경
307	Temporary Redirect	일시적 리다이렉션; 재요청 메서드 변경되지 않음

HTTP 상태 코드

- 일시적인 리다이렉션(temporary redirection)(2)
 - 상태 코드 302(Found)는 상태 코드 301(Moved Permanently)과 유사
 - 301(Moved Permanently)이 '요청한 자원이 완전히 다른 곳으로 이동했음'을 나타낸다면 302(Found)는 '요청한 자원이 임시로 다른 곳으로 이동했음'을 나타낸다는 정도의 차이
 - 상태 코드 302(Found)도 상태 코드 301과 마찬가지로 GET이 아닌 요청 메서드를 사용한 클라이언트가 상태 코드 302(Found)를 응답받을 경우, 두 번째 요청 메서드는 GET으로 바뀔 '수도' 있음



HTTP 상태 코드

- 일시적인 리다이렉션(temporary redirection)(3)
 - 307(Temporary Redirect)은 두 번째 요청 메서드를 변경하지 않는 상태 코드
 - 상태 코드 302(Found)의 애매모호함을 해결
 - 예) POST 요청을 보낸 클라이언트가 307을 응답받을 경우, 두 번째 요청 메서드도 POST로 유지
 - 상태 코드 303(See Other)은 두 번째 요청 메서드를 GET으로 바꿔 주기 위해 사용



HTTP 상태 코드

400번대: 클라이언트 에러 상태 코드

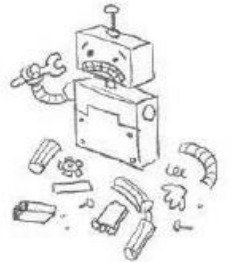
- '클라이언트에 의한 에러가 있음'을 알려 주는 상태 코드

상태 코드	이유 구문	설명
400	Bad Request	클라이언트의 요청이 잘못되었음
401	Unauthorized	요청한 자원에 대한 유효한 인증이 없음
403	Forbidden	요청이 서버에 의해 거부됨 (예: 접근 권한이 없을 경우)
404	Not Found	요청받은 자원을 찾을 수 없음
405	Method Not Allowed	요청한 메서드를 지원하지 않음

Google

404. That's an error.

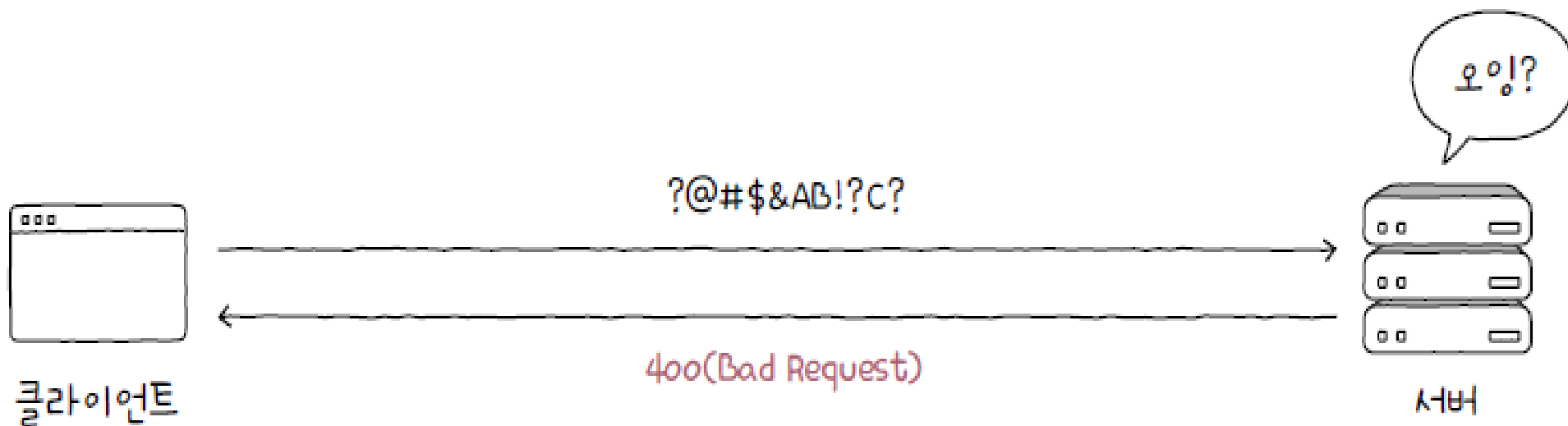
The requested URL
/badpage was not found on
this server. That's all we
know.



HTTP 상태 코드

◦ 상태 코드 400(Bad Request)

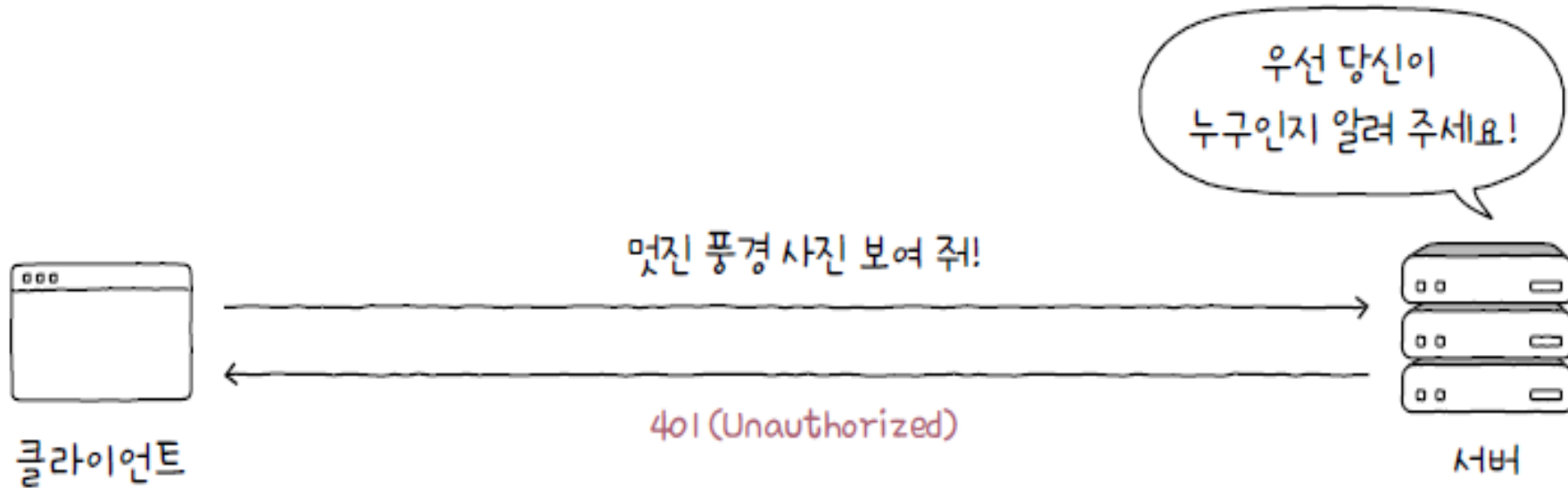
- 클라이언트의 요청이 잘못되었음을 알려 주는 상태 코드



HTTP 상태 코드

◦ 상태 코드 401(Unauthorized)

- 웹상에서 정보를 검색할 때 모든 자원에 접근이 가능한 것은 아니며 때로는 특정 자원에 접근하기 위해 인증이 필요
- 요청에 대한 인증이 필요할 경우 서버는 401(Unauthorized) 상태 코드를 응답

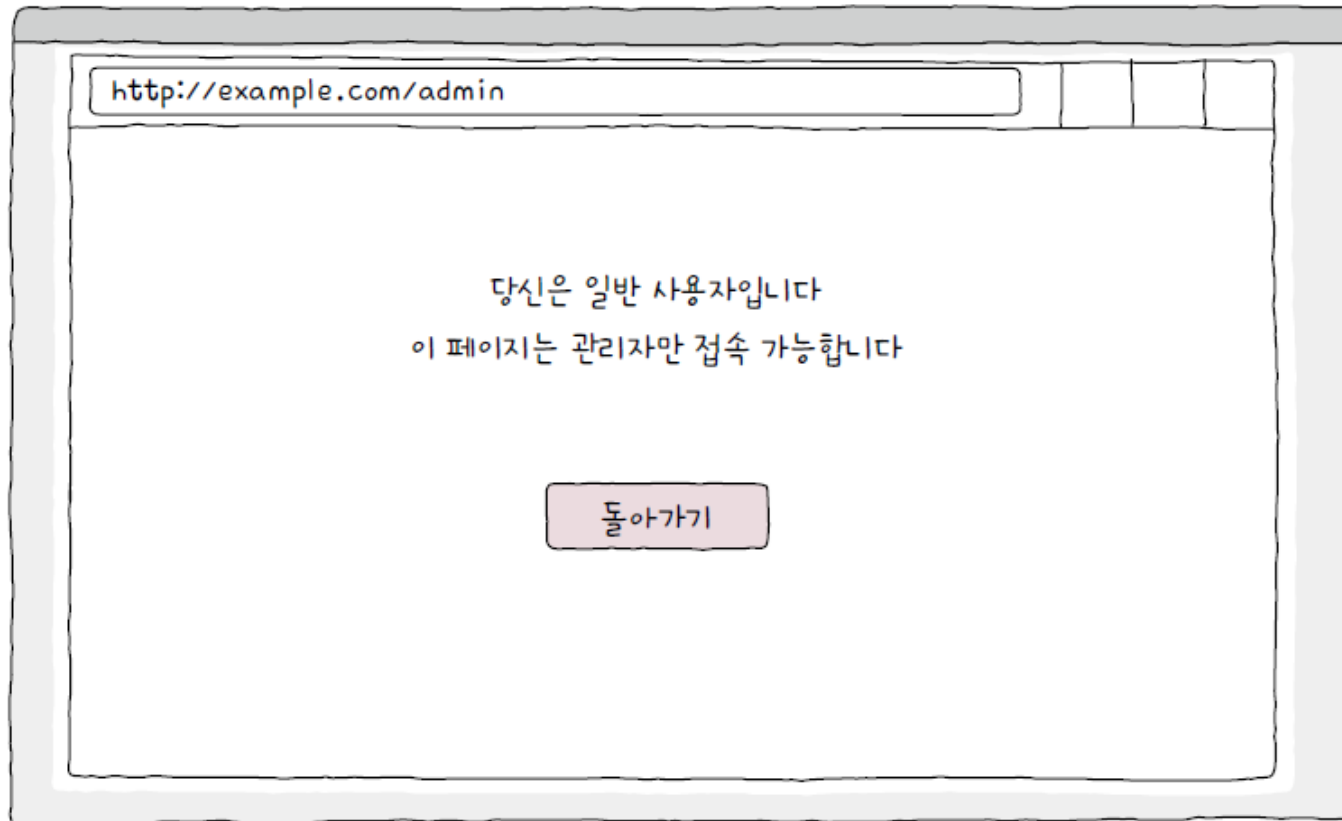


- 서버가 상태 코드 401(Unauthorized)로 응답할 때는 반드시 WWW-Authenticate라는 헤더를 통해 인증 방법을 알려 주어야 함

HTTP 상태 코드

◦ 상태 코드 403(Forbidden)

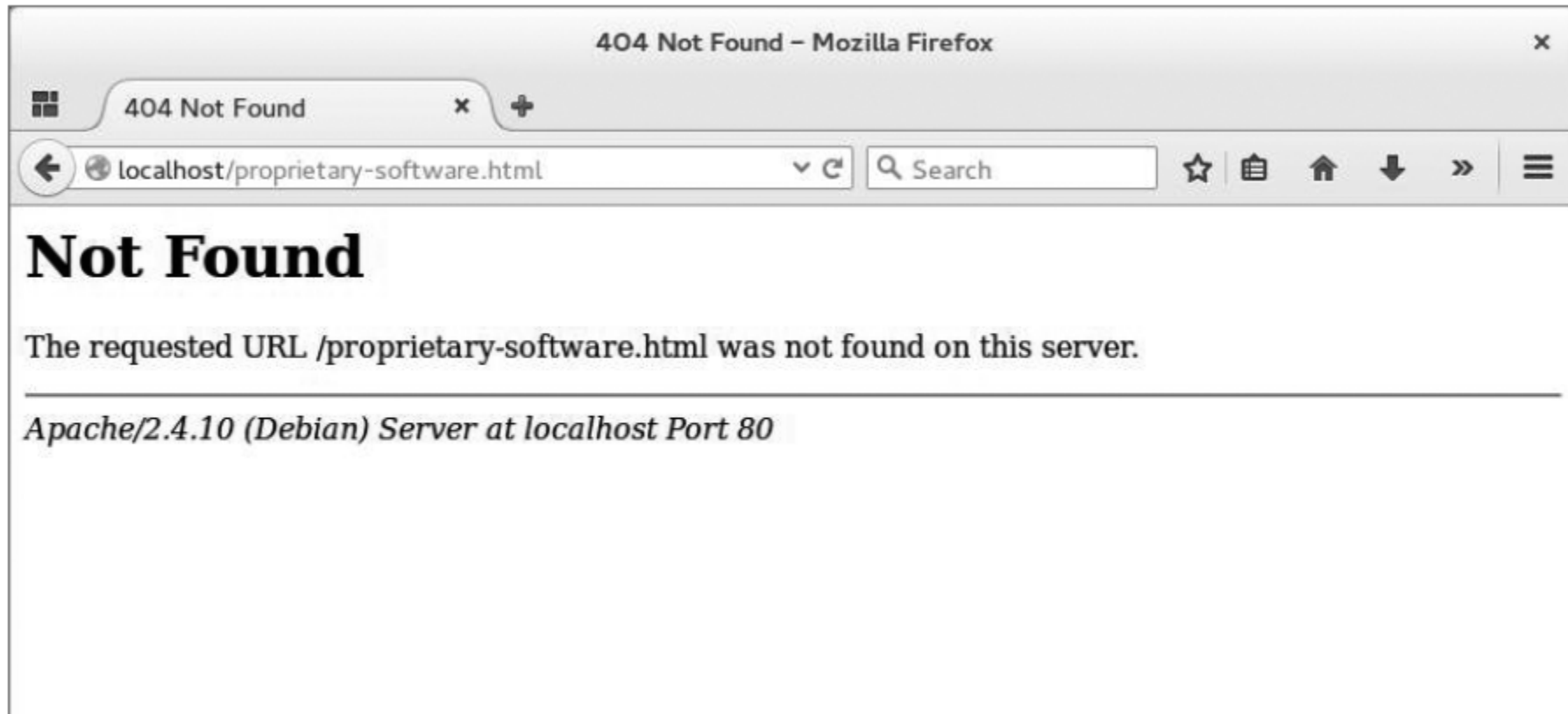
- 클라이언트의 권한이 충분하지 않다면 상태 코드 403(Forbidden)을 응답
 - 인증(Authentication) 여부와 권한 부여(Authorization) 여부는 다른 개념
 - 인증 - '자신이 누구인지 증명하는 것'
 - 권한 부여 또는 인가 - '인증된 주체에게 작업을 허용하는 것'



HTTP 상태 코드

◦ 상태 코드 404(Not Found)

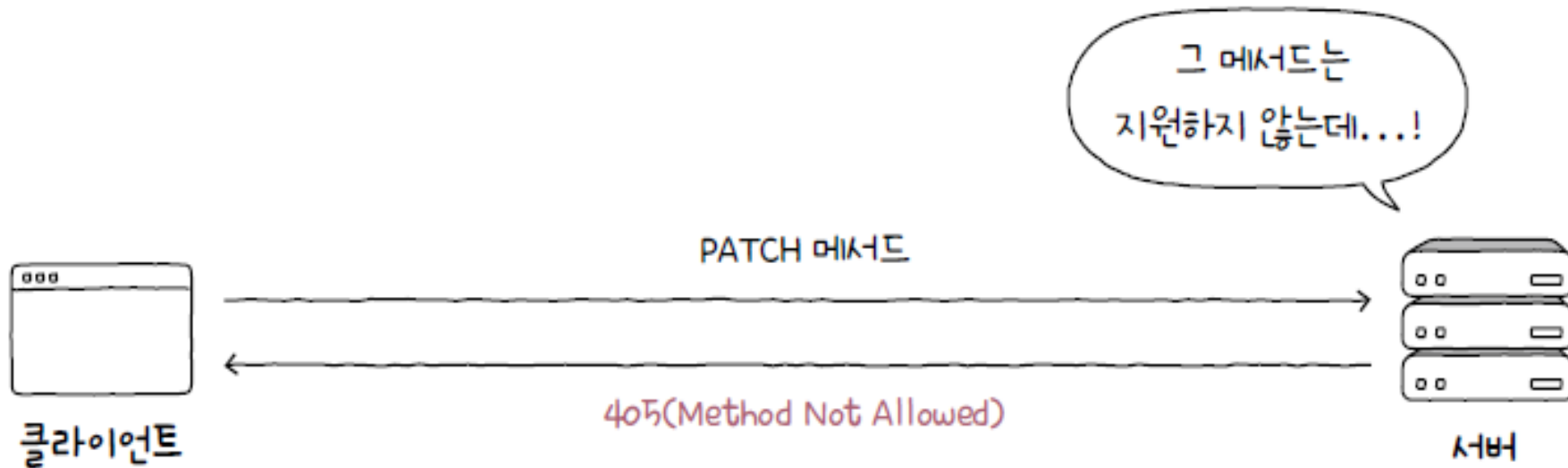
- 접근하고자 하는 자원이 존재하지 않음을 알리는 상태 코드
- 존재하더라도 공개하지 않는 자원에 대해 404(Not Found)를 응답하는 경우도 있음



HTTP 상태 코드

◦ 상태 코드 405(Method Not Allowed)

- 구현되지 않은 메서드로 요청을 보낸다면 상태 코드 405(Method Not Allowed)를 통해 해당 메서드의 미지원을 알림



HTTP 상태 코드

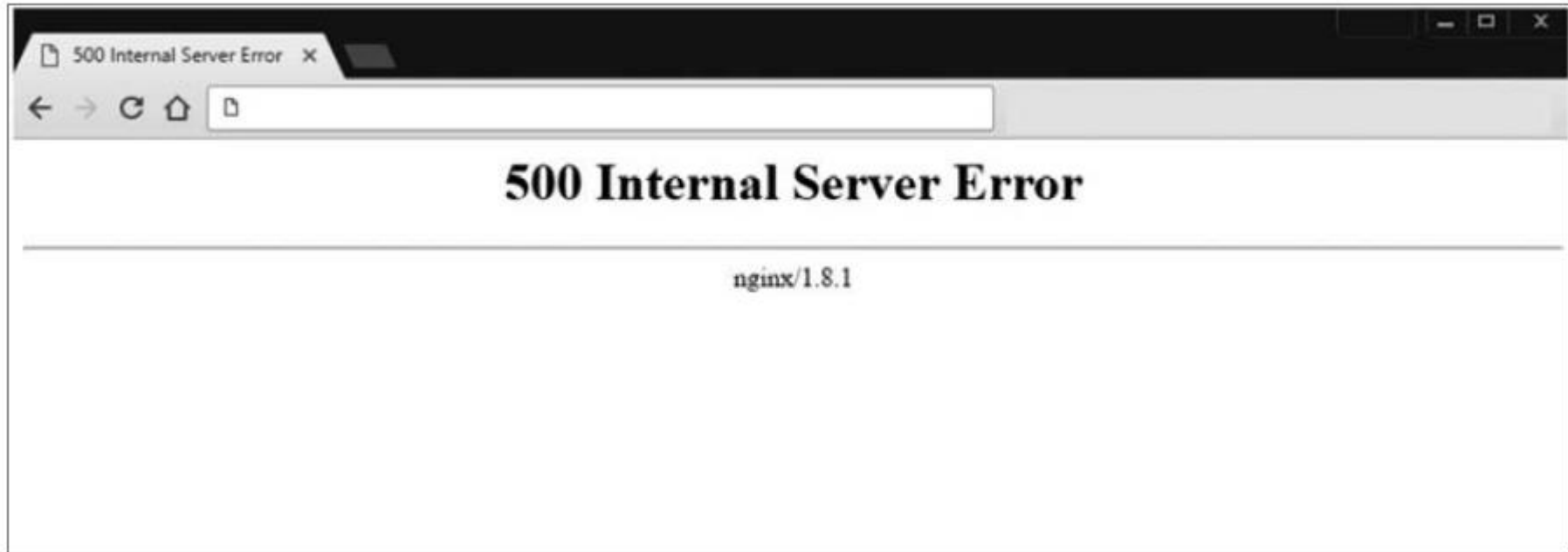
500번대: 서버 에러 상태 코드

- 500번대는 클라이언트가 올바르게 요청을 보냈을지라도 발생할 수 있는 서버 에러에 대한 상태 코드

상태 코드	이유 구문	설명
500	Internal Server Error	요청을 처리할 수 없음
502	Bad Gateway	중간 서버의 통신 오류
503	Service Unavailable	현재는 요청을 처리할 수 없으나 추후 가능할 수도 있음

HTTP 상태 코드

- 상태 코드 500(Internal Server Error)
 - '서버의 예기치 못한 상황으로 인해 요청을 처리할 수 없음'
 - 상태 코드 500(Internal Server Error)은 서버 내 에러를 통칭



HTTP 상태 코드

- 상태 코드 502(Bad Gateway)(1)

- 클라이언트와 서버 사이에 위치한 중간 서버의 통신 오류를 나타내는 상태 코드

Google

502. That's an error.

The server encountered a temporary error and could not complete your request.

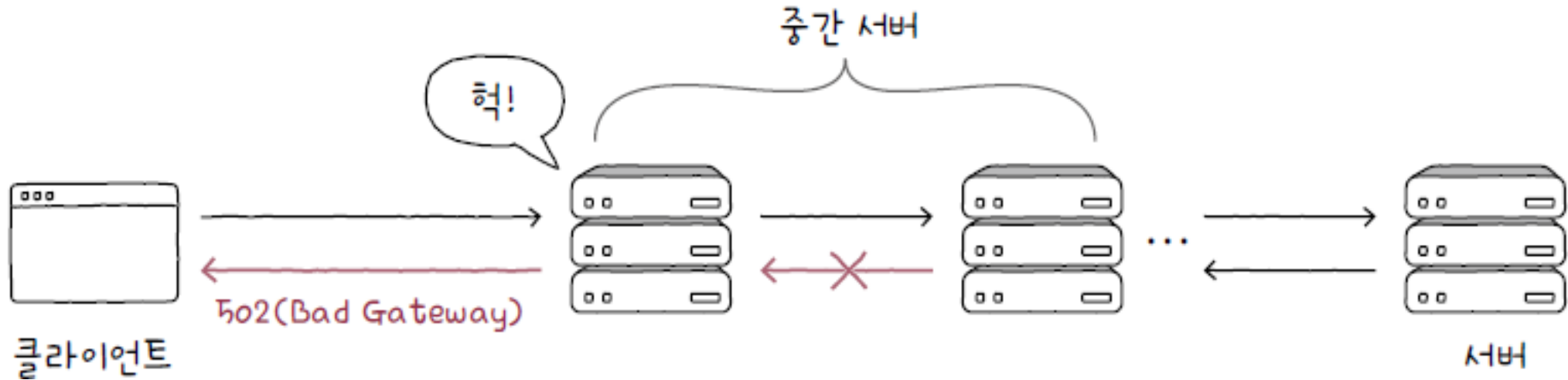
Please try again in 30 seconds. That's all we know.



HTTP 상태 코드

◦ 상태 코드 502(Bad Gateway)(2)

- 클라이언트와 서버 사이에 위치한 중간 서버가 유효하지 않거나 잘못된 응답을 받을 때



Chapter 05-2 HTTP(61) HTTP 상태 코드

- 상태 코드 503(Service Unavailable)
 - '현재 서비스를 일시적으로 이용할 수 없음'
 - 서버가 과부하 상태에 있거나 일시적인 점검 상태일 때 볼 수 있는 상태